
Practical AI for Academics

Claes Bäckman

Leibniz Institute for Financial Research SAFE

Three messages: agentic AI is ready, the workflow matters more than the model, and verification is now the bottleneck.

1 The landscape has shifted

AI tools have moved from **chat** to **reasoning** to **agentic** systems that read files, run code, and iterate.

Implication: tools you tried in 2023 are not the same tools today.

2 Value is in the workflow

The agent's leverage comes from **context** (your files), **standing instructions** (AGENTS.md), and **reusable skills** — not from raw model IQ.

Implication: a one-hour setup compounds across every later session.

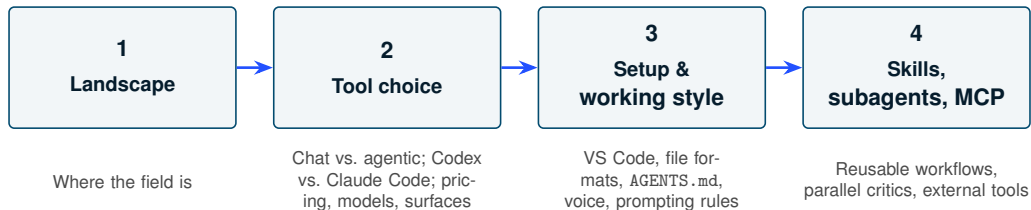
3 Verification is the new skill

Output volume is no longer the constraint — **judging it** is. Hallucinated citations, confident fabrication, and session drift are predictable failure modes.

Implication: the researcher's edge is critical reading, not typing speed.

Key takeaway. Pick one agent (Codex or Claude Code), set up VS Code, write a first AGENTS.md, and run one skill (\$review-paper) on a draft.

Morning agenda



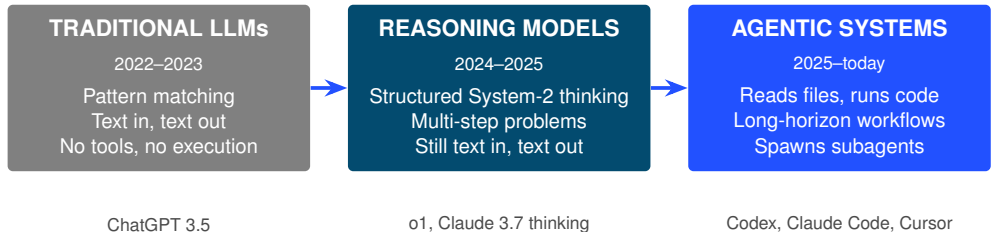
About the speaker. Claes Bäckman — housing, mortgages, household finance at SAFE.
Public resources: [Codex guide](#), [Codex vs. Claude Code](#), [researcher tooling](#).

Who has used ChatGPT or Codex inside an IDE or terminal?
Who has used the Codex app?

SECTION 1

The landscape

Three generations of AI models

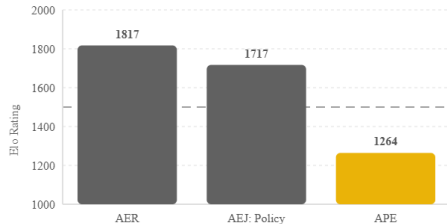


Key takeaway. A 2023 mental model (“ChatGPT is a chatbot that hallucinates”) no longer describes what the tools can do.

The pace is incredible

Evidence of the shift

- **Paul Novosad** wrote a paper with AI in ~ 3 hours, and then another one with the opposite result
- **Project APE** has automated $\sim 1,000$ policy-evaluation papers
- **López-Lira** runs an automated research pipeline on stock-return predictability
- **Novy-Marx and Velikov** (JEL paper) describes how you could automate academic research using LLMs



ELO ratings of papers by source.

AI is especially good at some parts of the work

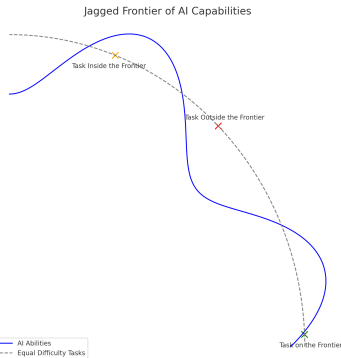
Coding is largely solved – Boris Cherny, head of Claude Code at Anthropic

Agents are world-class at some tasks, but embarrassing at others

The jagged frontier: Ethan Mollick, Dell'Acqua et al (2026)

- **Inside the frontier:** draft prose, code, lit-review summaries, structured critique, table re-derivation
- **Outside the frontier:** novel theory derivations, judgment calls on a fragile identification strategy, anything where the answer is not implicit in the training data

Key takeaway. Empirically map your own use cases: “trust, but verify”.



The frontier is uneven — map it before you trust it.

Agentic AI pays off across three distinct use cases

1 Research

- Lit-review summaries
- Pre-submission referee reports
- Feedback on your own drafts
- Responses to referees

Mental model: *AI as a feedback machine.*

2 Coding

- Stata / R / Python scripts
- Debugging error logs
- Robustness checks
- Replication packages

Mental model: *AI as a talented RA.*

3 Practical / admin

- Conference budgets and programmes
- CV updates and websites
- Email drafting, scheduling
- Sorting referee assignments

Mental model: *AI as a personal assistant.*

Admin is an undervalued example

Agents are great at reducing cost of administrative tasks

- **Friction is highest** — the starting cost dominates the doing
- **Quality bar is lower** — no referee, no audience
- **Nobody else to delegate to** — Small tasks not worth giving to someone else

Worked example — a CV for research grants

Different funders, different required formats.

Without an agent. ≈ 1 hours per funder. Reformat, re-order, drop sections, rewrite the bio.

With an agent. Drop the master CV and the funder's template in the folder. The agent reformats, you check.

Lots of time saved on a task no one was paying you to do.

Feedback was a scarce resource

The feedback gap

- ~40% of NBER papers acknowledge 3–5 RAs (Caro, 2024)
- Elite departments run internal seminars, lunch talks, drafts circulated to senior faculty
- Most researchers do not have access to that infrastructure
- Otherwise, feedback comes from referees — slow, sparse, end-of-process

What AI changes

- The same critic available for \$20/month, anywhere
- Pre-submission referee reports, identification critique, line-by-line edits on demand
- The PhD student at a regional university now has tools the senior MIT colleague has

Key takeaway. The biggest gain may not be **speed** but **access**. The field improves when more researchers get real feedback on their drafts.

AI is going to change how we do research

The obvious gains

Same research, done faster and better.

- **Faster coding** — Stata/R/Python drafts, debugging error logs, robustness checks, replication packages
- **Better writing** — structured critique, voice-matched edits, pre-submission referee reports

New ways of working

New outputs and projects that were infeasible before.

- **Different ways of communicating research** — interactive websites, policy briefs, dashboards, audio summaries from one paper
- **Different types of projects** — mass automation (Project APE), automated pipelines (López-Lira), text-heavy work that used to be too costly

Three things AI does not do

1. **It does not supply the question.** It lowers the cost of asking a question seriously; it does not tell you which question is worth asking. The research idea is still yours.
2. **It does not replace judgment.** Execution depreciates — identification, theory, and design do not. A weak IV plus a perfect agent is still a weak IV; the agent will execute it flawlessly anyway.
3. **The institutions have not caught up.** Disclosure norms do not yet distinguish AI-assisted from AI-generated. Peer review has not adapted. You will set your own standard before the journals tell you what theirs is.

Key takeaway. The earlier slides are bullish on capability. These three are why **verification** — not output — is the bottleneck for the next few years.

*Live demo: let's build a website from a conference transcript in
real time.*

Choosing your tool

A structural difference between Agentic tools and chat

Chat: ChatGPT, Claude.ai

Runs in a sandbox.

- You describe the problem in words
- Paste text in, get text out
- No access to your files or your code
- Output is generic — surface-level

When to use. Brainstorming, draft polish on a single paragraph, quick lookups.

Agentic: Codex, Claude Code

Runs on your computer.

- Reads every file in your project folder
- Edits files, runs code, calls shell commands
- Calls tools: search, web, MCP servers
- Output is grounded in **your** files

When to use. Any task that touches more than one file or needs to execute code.

Key takeaway. Worked example — asking for a conference budget: chat returns a generic template; Codex finds the keynote speakers, the date, the dinner, and the PhD travel funding by reading the conference folder.

Codex vs. Claude Code: the workflow is the same.

Both will: reads your files, runs code, edits drafts, iterates, spawns subagents.

Codex (OpenAI)

Project memory: AGENTS.md

Skills trigger: \$review-paper

Backed by: GPT-5 family

Surfaces: VS Code, CLI, web, mobile

Claude Code (Anthropic)

Project memory: CLAUDE.md

Skills trigger: /review-paper

Backed by: Opus / Sonnet / Haiku

Surfaces: VS Code, CLI, web, desktop

Key takeaway. Switching costs are small once you understand the pattern. Today's examples use **Codex**

Three pricing tiers cover almost every academic — start at Plus.

Free	\$0	Casual chat; very limited Codex usage	
Plus	\$20 / mo	Light daily Codex use, projects, deep research	▶ Start here
Pro	\$100–200 / mo	Heavy Codex use; larger files and context budgets	
API	Pay per token	Automation, batching, custom-built tooling	

Key takeaway. Rule of thumb: start on Plus; move to Pro when Codex, deep research, or file limits start interrupting real work. Use API only if you are building something programmatic.

Three model tiers

Tokens are the unit models bill and reason in — about 3–4 characters of English. A short paper is $\approx 15\text{--}30\text{k}$ tokens. A log file can blow past 100k. You can switch the model per task inside Codex.

Fast / cheap

Haiku, GPT-5 mini

Conversions, summaries, bulk tasks

Default

Sonnet, GPT-5

Daily coding, editing, drafting

Frontier

Opus, GPT-5 Pro

Hard reasoning, long context, adversarial review

Key takeaway. Start on the default. Drop down for bulk tasks or to stretch a weekly token limit; step up to frontier for planning, complex reasoning, or adversarial review.

You can use whatever program you like

Surface	What it is good for
VS Code	Recommended. File tree, diff view, integrated terminal, side-by-side PDF preview.
Terminal / CLI	codex or claude in any shell. Good for servers and quick one-shots.
Desktop app	Codex desktop or Claude Desktop <i>Cowork</i> / <i>Code</i> tabs.
Web / cloud	chatgpt.com/codex or claude.ai/code . Point at a GitHub repo.
Mobile	Capture ideas; monitor or redirect cloud agents. Not for editing.
Other editors	Cursor, JetBrains, Zed — keep your editor, add the agent.

Key takeaway. Pick **VS Code** if you want to see and edit the differences. More detail at claesbackman.com/agentic-ai-overview.

Codex has a dedicated app for Mac and Windows

You open the app and direct it at a folder

Then the agent has context and you can ask it to do all the same tasks as in VS Code

- You can see the “diffs” in the side-panel

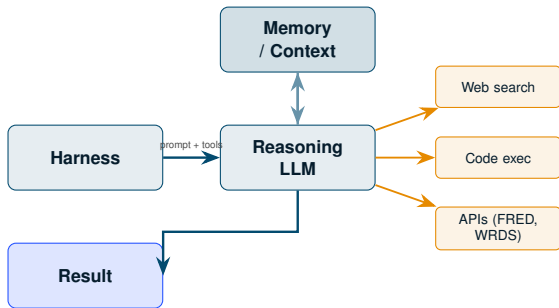
Drawback is that you cannot see what the edits are as easily

What have you used ChatGPT or agentic AI for so far?

SECTION 3

How the tools work

The agent is the harness around the model



What happens when you type a prompt

1. Your prompt goes to the model with AGENTS .md, recent files, and history
2. The model decides which **tools** to call — read, edit, shell, web
3. Each tool returns a result back into the conversation
4. The loop continues until the task is done — or the agent asks you a question

Key takeaway. ChatGPT, Claude.ai, Codex, Claude Code, and Cursor are all harnesses around **similar** models. The harness is the source of capability.

Codex works through six core capabilities

Capability	What it lets the agent do
Project files	Read files, inspect folders, understand project structure
Editing	Propose and apply diffs; create or modify files
Shell	Run commands such as <code>Rscript</code> , <code>pdflatex</code> , tests, linters
Search	Search filenames and file contents in the project
Web	Search or fetch web pages when web access is enabled
MCP / plugins	Connect to external tools: GitHub, databases, custom servers

Setting up your workspace

Five steps get VS Code + Codex working in under five minutes.

1. Open VS Code and the Extensions panel (Ctrl+Shift+X / Cmd+Shift+X)
2. Search “**Codex**” and install *Codex — OpenAI’s coding agent* (extension ID `openai.chatgpt`)
3. Restart VS Code — Codex opens in the right sidebar
4. Sign in with your ChatGPT account (Plus/Pro/Business/Edu/Enterprise all include Codex) or an API key
5. Open your **full project folder** (File > Open Folder) — never a single file

What an IDE actually is. Editor + file explorer + terminal + extensions, in one window. VS Code is the dominant free one. Think of it as Word or R Studio for code, LaTeX, markdown, and plain text.

A handful of extensions cover Stata, Python, LaTeX, and data

Stack	Extension ID	What it does
Stata	<code>tmonk.stata-workbench</code>	Run Stata from the editor; MCP server lets the agent execute commands
Python	<code>ms-python.python</code>	Syntax, IntelliSense, run button
LaTeX	<code>james-yu.latex-workshop</code>	Compile on save, side-by-side PDF preview, inline errors
Data	<code>mechatroner.rainbow-csv</code>	Colour-coded CSV columns; SQL-like filtering
	<code>tomoki1207.pdf</code>	Renders PDFs inline — read codebooks next to your code

Key takeaway. Install Python even if you do not write it. The agent uses Python as glue for many different tasks.

*What is one task from this week that you would hand to an agent
— if you trusted it to do the work?*

Working with the agent

Every Codex session follows the same pattern — open the folder, set context, then prompt.

1. Open VS Code and the project folder (File → Open Folder)
2. Open the Codex panel in the right sidebar
3. Confirm Codex is pointed at the **full project folder** — not a single file
4. Send a first orienting prompt

First prompt to try:

“Read the files in this folder and tell me what this project is about.”

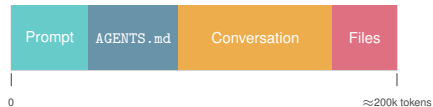
Expected behaviour. Codex reads the folder, summarises the structure, and asks you what you want to do next. **If it does not ask a question, your AGENTS.md should tell it to.**

Context is the budget that you have available

Habits that protect it

- Point at specific files, don't let Codex read everything
- Start a new session for a new task (+ button)
- Watch the context indicator
- Convert long PDFs to markdown once, up front

Anatomy of a context window



File formats fall into three buckets

● **Native — no friction** `.md, .tex, .txt, .bib, .csv, .json, .yaml`

● **Code — native** `.py, .R, .do, .ipynb`

● **Usable — convert first** `.docx, born-digital .pdf, .xlsx`

● **Hard — OCR or skip** Scanned PDFs, `.dta / .rds` binaries, `.qsf`

Key takeaway. Convert once, commit the markdown alongside the original, point the agent at the markdown. Re-converting on every session is the single biggest waste of context.

AGENTS.md is the standing-instructions file — write it once, stop repeating yourself.

About me

I am a researcher in [field]. I write papers, lecture notes, policy pieces.

How I want you to work with me

- Ask clarifying questions before generating long output.
- Keep a skeptical, critical tone in feedback. Do not flatter.
- When editing, preserve my voice. Avoid generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality when summarising empirical work.
- Do not insert emoji or markdown decorations in formal documents.

Three files hold your durable project context — know what goes where.

File	Use it for
<code>AGENTS.md</code>	Project instructions. Dataset structure, sample restrictions, paper title, coauthor conventions, which script produces which output.
<code>voice.md</code>	Writing samples. Paragraphs, abstracts, and tone notes that Codex should match when editing prose.
<code>.codex/skills/...</code>	Reusable workflows. Named tasks you want to run again without rewriting the prompt.

Key takeaway. Keep secrets and raw confidential data **out of all three** — these files are read into context on every session and may be sent to the model provider.

Bootstrap AGENTS.md in two minutes — let Codex write the first draft.

Prompt:

“Read this project folder and draft an AGENTS.md file with the project structure, conventions, and instructions you should follow. Do not edit files yet.”

What Codex does

- Reads the folder
- Infers languages, structure, conventions
- Generates an AGENTS.md draft

Then you edit it. Add tone preferences, things to avoid, sample restrictions Codex could not infer.

*What three standing instructions would you put in your own
AGENTS.md?*

A `voice.md` file makes edits sound like you

When editing my prose, match the voice in `voice.md`.
Do not use em-dashes, bullet lists, or passive constructions unless the original does.
Do not use "it's not x, it's y"-style writing.

What goes in it

- 5–10k words of your prose — intros, abstracts, paragraphs you are proud of

We will build one this afternoon using the `voice-extractor` skill.

Key takeaway. Reference it from `AGENTS.md` to give Codex writing context for every session

Ask Mode — think before you write. The agent plans, you correct, then it executes.

For anything non-trivial, ask Codex to outline its approach **before** generating.

Example prompt:

“Before writing the policy brief, outline the structure, the three findings you plan to lead with, and the one caveat you’ll flag. Do not write the brief yet. Ask me questions if you need to.”

You review the plan, correct the framing, then ask for execution. Codex supports this directly:

- **Ask Mode** — planning, no edits
- **Code Mode** — execute the agreed plan

Key takeaway. Planning catches the wrong approach when it is cheap to fix — before the agent has written 800 lines you have to read.

Permissions follow a three-tier model

Read-only

Can inspect files; cannot write.

Workspace-write

Can edit files inside the project folder.

Full access

Read/write broadly, run with fewer restrictions. *Use rarely.*

Approvals control when Codex must ask before running commands or leaving the sandbox.

Key takeaway. Keep approvals conservative for academic work.

Five prompting rules: specific, iterate, correct early, verify, restart.

1. **Specific.** “Feedback on my paper” is a worse request than “Give me feedback on the identification strategy in Section 4.”
2. **Iterate.** 10–20 turns, not one.
3. **Correct early.** If the first answer is off, fix it now — or restart the chat.
4. **Trust but verify.** Run the code. Re-read the passage. Check the citation.
5. **Open new sessions.** 30+ turns or the task has shifted — `/clear` and restart.

Key takeaway. The researcher who sends one long prompt and accepts the answer gets generic output. The researcher who treats the agent like a junior colleague and argues for ten turns gets work product.

Four failure modes to recognise — flattery, generic feedback, fabrication, drift.

Failure mode	Symptom	Fix
Flattery	<i>"This is a well-structured piece."</i>	Ignore. Ask for the strongest objection a hostile referee would raise.
Generic feedback	<i>"Consider strengthening the motivation."</i>	Push back: "Give me a specific sentence that would make it stronger."
Confident fabrication	Codex describes a passage that is not in the draft, or cites a paper that does not exist.	Always re-read the passage. Always check the citation.
Session drift	Repeats mistakes, invents files, gives circular answers, forgets the last three turns.	Start a fresh session. Cost: 5 seconds.

Working with secure data

Secure servers split the workflow

What Codex can still help with on your laptop

- Your `.do` files
- Writing and iterating on empirical strategy

Codex cannot run files or check the logs, but can still be very helpful in a number of ways

Two artifacts to keep local

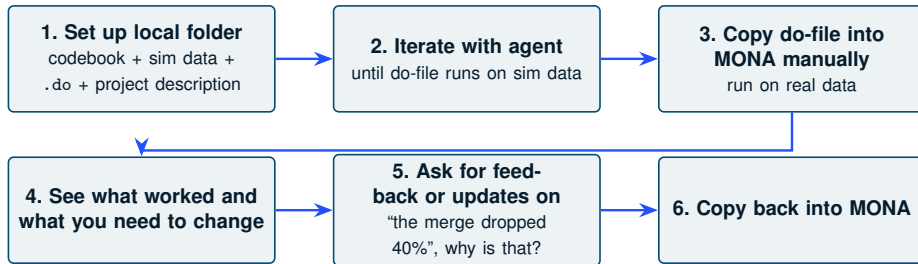
1 Codebook in markdown

- Convert once (`$pdf-to-markdown`)
- Variable names, labels, value ranges, missing-value codes
- Agent reads it before writing `recode`, `label` `define`, `merge`

2 Simulated dataset

- Same variable names, types, ranges — not real values
- AER/JF/ReStud replication packages often ship with these
- Or generate from the codebook

A workflow for secure data on servers



Key takeaway. The agent never sees the real data — but it sees everything that explains the data. That is usually enough to write correct Stata code.

Skills, subagents, and MCP

What is a prompt you have typed more than twice this month?

A skill is a markdown file the agent loads on demand — invoke with \$name.

A skill lives at `.codex/skills/your-skill/SKILL.md`:

name: review-paper

description: Run a 6-agent pre-submission referee report for an academic paper targeting a specified journal

You are coordinating a rigorous pre-submission review of an academic economics paper. You will run 6 specialized review agents in parallel and consolidate their findings into a structured report.

Invoke with `$paper-feedback`. Codex loads the instructions and runs them.

Two slots, different jobs — AGENTS.md is always loaded, skills are on demand.

AGENTS.md — always loaded

- Loaded into context every session
- Background, conventions, voice
- “How I want you to work with me”

Cost: *permanent context budget.*

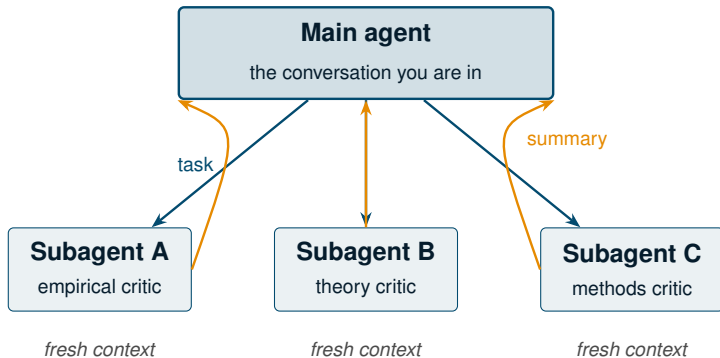
Skill — on demand

- Loaded only when invoked with \$name
- A specific workflow, with structured output
- “Do this thing for me”

Cost: *zero until triggered.*

Key takeaway. If you find yourself pasting the same prompt multiple times, it should be a skill.

Subagents add two things the main agent cannot do alone — parallelism and context isolation.



Key takeaway. The \$review-paper skill we will see this afternoon spawns **six** subagents.

The second-opinion pattern — spawn an adversary to check on the work.

The main agent sees its own prior answer and has reason to defend it.

Why a fresh subagent is different

- No chat history: no anchoring on what you already wrote
- No conversational momentum: no reason to agree with the main agent
- Clean context: can hold the whole artifact at once

When to spawn one

- Before accepting a long answer: *“is this actually right?”*
- After a long refactor: *“what did we break?”*
- When you suspect flattery: *“give me the honest version”*
- Before sending: *“read this as a hostile reader”*
- After doing automatic edits: *“look over the changes we did”*

Key takeaway. The phrases “**spawn a subagent**” or “**launch an agent**” are the trigger words. The main agent knows what to do and will write the instructions for you.

Live demo — let's launch a subagent to criticise the website we just built.

Three patterns cover most adversarial-review use cases.

1. Critique a draft as a hostile referee

“Launch an agent to read `draft.tex` and give me the three strongest objections a referee would raise.”

2. Verify numbers against code

“Spawn a subagent to re-derive the numbers in Table 3 from `05_results.do` and tell me if they match the draft.”

3. Stage an internal debate

“Spawn two subagents: one argues the identification strategy works, one argues it doesn’t. Then summarise the disagreement.”

Key takeaway. You do not need to write the subagent’s instructions. The main agent does that — you only set the framing.

MCP servers plug the agent into external systems — know it exists, install only with a clear use case.

Model Context Protocol. A standard for letting Codex call external tools beyond the local filesystem.

Examples already in production

- GitHub — open PRs, read issues, comment on code review
- Databases — query Postgres, BigQuery, etc. directly
- Custom servers you wrote yourself

Like browser extensions, but for the agent. The list is growing fast.

Key takeaway. For most academics, today, **you do not need MCP**. Know the concept exists; install one only when you have a concrete use case.

Lunch

See you back at the start of the afternoon session.

Appendix: AGENTS.md starter template

About me

I am a researcher in [field].

How I want you to work with me

- Ask clarifying questions before generating long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md). No generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality.
- Do not insert emoji or markdown decorations in formal docs.

Appendix: My skill library

Skill	Use
<code>\$review-paper</code>	Pre-submission referee report, 6 agents
<code>\$review-paper-light</code>	Fast 2-agent version, $\approx$ 1 minute
<code>\$review-paper-code</code>	Paper + code alignment review
<code>\$review-grant</code>	Pre-submission grant review, 6 agents
<code>\$review-pap</code>	Pre-analysis-plan review
<code>\$paper-version</code>	Paper $\rightarrow$ policy brief / 1-page / 5-page
<code>\$pdf-to-markdown</code>	Convert PDFs to readable markdown
<code>\$done</code>	Session capture and log

Appendix: signs of session drift

- Codex ignores AGENTS.md
- Repeats mistakes you already corrected
- Invents files that do not exist
- Gives circular answers
- Forgets the last three turns

Fix. Start a fresh session or close and reopen. Re-state the task. Re-attach the relevant file.

30+ turns or the task has shifted → start fresh. Always.